

Week 1 · 실습 워크북: 첫 리소스 apply → state → destroy

📌 참고

이 문서는 실습 파트(35분)입니다. 개념은 [개념워크북.pdf](#) 를 먼저 보세요. [실습] 은 직접 치고, [확인] 은 눈으로 대조하고, [합정] 은 미리 피해야 할 실수이고, [관찰 🍷] 은 빈칸을 채웁니다.

오늘 실습을 마치면: 리소스를 코드로 `apply` 해서 만들고, `state` 로 확인하고, `destroy` 로 지울 수 있다.

오늘 만들 것

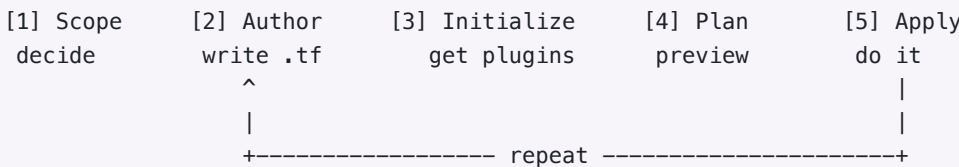
S3 버킷 1개 + 퍼블릭 접근 차단 + (심화) 버전 관리. 빈 버킷은 사실상 \$0이라 비용 위험이 없습니다.

! 꼭 기억

작업 폴더는 이 리포의 `practice/` 하나뿐입니다. 새 폴더를 만들지 마세요. `practice/` 에는 이미 `versions.tf` · `providers.tf` · `variables.tf` 가 준비돼 있고, 여러분은 `main.tf` 와 `outputs.tf` 의 `# TODO` 만 채웁니다.

오늘의 진행 순서: 배포 5단계

HashiCorp 공식 문서는 Terraform으로 인프라를 배포하는 과정을 5단계로 정의합니다. 오늘 입력하는 명령은 전부 이 다섯 단계 중 하나에 해당합니다. **지금 내가 몇 번째 단계에 있는지** 알면 진행 상황을 놓치지 않습니다.



단계	하는 일	오늘 어디서
1. Scope	뭘 만들지 정한다	"오늘 만들 것" · A-1
2. Author	<code>.tf</code> 파일에 적는다	A-2 ~ A-5 · A-7 · B-3
3. Initialize	provider(플러그인)를 설치한다	A-6
4. Plan	뭐가 바뀔지 미리 본다	A-8 · B-3
5. Apply	실제로 반영한다	B-1 · B-3 · B-4

한 번 5단계를 다 돌고 나면, 그 뒤로는 2 → 4 → 5(Author → Plan → Apply)만 계속 반복합니다. `init` 은 보통 처음 한 번뿐입니다. Block B에서 이 반복을 두 번 더 돌립니다.

참고

`destroy` 는 이 5단계에 없습니다. 5단계는 "만들기"까지의 흐름이고, `destroy` 는 라이프사이클의 끝(Day n)입니다. 그래도 오늘 세션은 `destroy`까지 해야 끝납니다.

Block A. 워크스루 (8분): 강사와 함께

5단계 중 Scope → Author → Initialize → Plan

[실습] A-1. 도구 · 자격증명 확인 (Scope)

```
terraform version      # 1.9.0 이상이어야 함
aws --version          # aws-cli 설치 확인
aws sts get-caller-identity # 내 자격증명이 실제로 동작하는지 확인
```

```
Terraform v1.9.0
on darwin_arm64
```

숫자는 사람마다 다릅니다. 1.9.0 이상이기만 하면 됩니다(`practice/versions.tf` 가 그렇게 요구합니다).

팁

셋 중 하나라도 `command not found` 가 나오면 설치가 안 된 겁니다. 설치는 여기를 그대로 따라 하세요 → [Install Terraform \(HashiCorp 공식\)](#). 이 시간에 설치부터 하면 8분이 모두 소요되므로, 설치가 안 된 사람은 Discord에 남기고 다른 참가자 화면을 보며 따라오세요.

[확인] 자격증명 OK

`get-caller-identity` 가 이런 JSON을 출력하면 성공입니다.

```
{
  "UserId": "AIDAXXXXXXXXXXXXXXXXXX",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/boaz-tf-study"
}
```

이 세 값을 기억해두세요. A-4에서 Terraform이 똑같은 세 값을 직접 조회하도록 코드를 작성합니다.

[함정] 자격증명이 인식되지 않을 때

에러	원인	해결
Unable to locate credentials	자격증명 미등록	aws configure 로 AccessKey/Secret/ ap-northeast-2 등록
ExpiredToken / AccessDenied	키 만료·비활성	IAM에서 재발급. 루트 계정 키는 쓰지 말 것

[실습] A-2. 이미 있는 코드 읽기 (Author 준비)

```
cd practice
ls
```

.tf 파일이 다섯 개 보입니다. Terraform은 현재 폴더의 .tf 파일을 전부 읽어서 하나로 합칩니다. 파일 이름도, 순서도 상관없습니다. 한 파일에 모두 작성해도 동작하지만, 나눠 두면 사람이 찾기 쉬워서 이렇게 나눈 것입니다.

파일	무슨 일을 하나	오늘 수정하나
versions.tf	Terraform 자신과 provider 버전을 고정	읽기만
providers.tf	AWS provider 동작 옵션(리전)	읽기만
variables.tf	입력 변수 선언	읽기만
main.tf	리소스·데이터 소스	채운다
outputs.tf	출력값	채운다

먼저 versions.tf 입니다. 이 안의 terraform { } 블록은 인프라가 아니라 Terraform 자신을 설정하는 유일한 블록입니다.

```
terraform {
  required_version = ">= 1.9.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

source : 어느 provider를 받을 것인가. "hashicorp/aws" 는 축약형이고, 원래 주소는 registry.terraform.io/hashicorp/aws 입니다.

```
registry.terraform.io / hashicorp / aws
      host      namespace  name
```

(omitted by default) (who made it)

호스트는 생략하면 공식 레지스트리가 기본값이라 두 조각만 적어도 됩니다. `namespace` 가 중요한데, `hashicorp/aws` 와 다른 사람이 올린 `someone/aws` 는 완전히 다른 provider입니다.

version : 어느 버전까지 허용할 것인가. 버전 제약을 적지 않으면 Terraform은 **항상 최신 버전**을 받습니다. 테스트하지 않은 버전이 예고 없이 설치될 수 있습니다. 그래서 허용 범위를 명시합니다.

표기	허용 범위	안 받는 예
<code>~> 6.0</code>	6.0 이상 7.0 미만 (6.x 전부)	<code>7.0.0</code>
<code>~> 6.1.0</code>	6.1.0 이상 6.2.0 미만	<code>6.2.0</code>
<code>>= 1.9.0</code>	1.9.0 이상 전부	<code>1.8.5</code>

`required_version` 은 provider가 아니라 Terraform 본체의 버전 제약입니다. A-1에서 `terraform version` 을 친 이유가 이것입니다.

📌 참고

HashiCorp 공식 튜토리얼은 이 블록을 `terraform.tf` 라는 파일에 두라고 권합니다. 이 레포는 `versions.tf` 라고 부릅니다. 둘 다 혼한 관례이고 동작은 완전히 같습니다. Terraform은 파일 이름을 구분하지 않습니다.

다음은 `providers.tf` 입니다. `terraform` 블록이 "무엇을 설치할지"라면, `provider` 블록은 "설치한 것을 어떻게 사용할지" 입니다.

```
provider "aws" {  
  region = var.region  
}
```

`provider "aws"` 의 `aws` 는 `versions.tf` 의 `required_providers` 안에 있던 그 `aws` 와 같은 것을 가리킵니다. 여기 적은 `region` 은 이 provider가 만드는 모든 리소스에 적용됩니다.

🚫 절대 금지

자격증명은 코드에 절대 넣지 않습니다. AWS provider는 AWS CLI와 똑같은 방식으로 인증합니다. 따라서 `aws` `configure` 로 등록한 값을 자동으로 찾아 사용합니다. AccessKey를 `.tf` 나 `.tfvars` 에 적으면 커밋하는 순간 그대로 유출됩니다.

[실습] A-3. 내 변수 파일 만들기 (Author)

`variables.tf` 는 "이런 변수를 받는다"는 선언이고, 실제 값은 `terraform.tfvars` 에 씁니다.

```
cp example.tfvars terraform.tfvars
```

terraform.tfvars 를 열어 project_name 을 본인 GitHub ID로 바꿉니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w1-{본인-github-id}"    # 예: boaz26-w1-kdh1834
```

💡 팁

파일 이름이 정확히 terraform.tfvars 면 Terraform이 자동으로 읽습니다. 그래서 -var-file 같은 옵션이 필요 없습니다. 이름을 my.tfvars 로 바꾸면 자동으로 읽히지 않으므로 그대로 두세요.

⚠️ 주의

S3 버킷 이름은 전 세계에서 유일해야 합니다. 버킷 이름 규칙은 소문자·숫자·하이픈만, 3~63자입니다. 대문자와 언더스코어(_)는 InvalidBucketName 에러입니다.

최종 버킷명 뒤에 계정번호 12자리가 붙으므로(A-4), project_name 은 40자 이내로 잡으세요.

! 꼭 기억

CP1: 정한 project_name 을 Discord에 한 줄로 올려주세요. 오타를 여기서 확인합니다.

[실습] A-4. 데이터 소스: 계정 번호를 코드가 직접 조회하게 하기 (Author)

main.tf 의 첫 번째 TODO입니다. 중괄호 안이 비어 있는 게 맞습니다.

```
data "aws_caller_identity" "current" {}
```

A-1에서 친 aws sts get-caller-identity 를 기억하세요. 이 한 줄이 정확히 그 명령입니다. 다만 이번엔 내가 터미널에서 입력하는 것이 아니라, Terraform이 plan을 만들 때 AWS에 조회합니다. 반환되는 값도 그때 본 것과 같습니다: account_id , arn , user_id .

```
data "aws_caller_identity" "current" {}
|      |
|      | +--- name (my own label)
|      +----- type (given by the aws provider)
+----- block kind

reference: data.aws_caller_identity.current.account_id
```

resource 와 생김새는 똑같은데 앞에 data 가 붙습니다. 차이는 딱 하나입니다.

블록	하는 일	AWS를
<code>resource</code>	내가 만들고 관리한다	바꾼다 (쓰기)
<code>data</code>	이미 있는 것을 조회한다	바꾸지 않는다 (읽기)

참조할 때 `data.` 로 시작하는 것도 잊지 마세요. `resource` 는 `aws_s3_bucket.lab` 이지만, `data` 는 `data.aws_caller_identity.current` 입니다.

왜 이걸 쓰나? 계정 번호를 `terraform.tfvars` 에 직접 적어도 되긴 합니다. 그런데 직접 적은 값은 ① 오타가 나고 ② 다른 계정에서 이 코드를 돌리면 틀린 값이 되고 ③ 커밋하면 기록으로 남습니다. **조회할 수 있는 값은 하드코딩하지 않고 조회한다.** 이것이 데이터 소스를 쓰는 이유입니다.

📌 참고

AWS 계정 번호(12자리)는 비밀번호가 아닙니다. 공개돼도 그걸로 로그인할 수는 없습니다. 다만 공개적으로 공유할 값도 아니므로, Discord에는 버킷 전체 이름 말고 `project_name` 만 올리세요. 참고로 `이름-계정번호` 형태의 버킷명은 실무 표준 패턴입니다(AWS CDK가 부트스트랩 버킷 이름을 이렇게 만듭니다).

[실습] A-5. 첫 리소스 작성 (Author)

`main.tf` 의 두 번째 TODO를 채웁니다.

```
resource "aws_s3_bucket" "lab" {
  bucket = "${var.project_name}-lab-${data.aws_caller_identity.current.account_id}"

  tags = {
    Project = var.project_name
    Study   = "boaz-terraform-26"
    Week    = "1"
  }
}
```

`bucket` 값 한 줄 안에 변수(`var.`)와 데이터 소스(`data.`)가 같이 들어갔습니다. 결과는 `boaz26-w1-kdh1834-lab-123456789012` 같은 모양이 됩니다. 계정이 다르면 이름이 자동으로 달라지므로 버킷명 충돌이 구조적으로 발생하지 않습니다.

```
resource "aws_s3_bucket" "lab" {
|           |           |
|           |           +--- resource name  (my label, never sent to AWS)
|           +----- resource type   (the aws_ prefix = the provider)
+----- block kind

address:  aws_s3_bucket.lab
```

- 리소스 타입 `aws_s3_bucket` : provider가 정합니다. 앞의 `aws_` 가 provider 이름입니다.
- 리소스 이름 `lab` : 내가 붙이는 코드 내 이름. AWS에는 전달되지 않습니다.

- 이 둘을 합친 `aws_s3_bucket.lab` 이 리소스 주소(resource address) 입니다. `state list` 에 뜨는 것도, 다른 리소스에서 참조할 때 쓰는 것도 이 주소입니다.
- `bucket` , `tags` 는 인자(argument): 실제로 AWS에 전달되는 값입니다.

⚠ 주의

`bucket` (AWS에 등록되는 실제 이름)과 `lab` (코드 안에서만 쓰는 이름)은 **완전히 다른 것**입니다. 이 둘을 혼동하는 것이 1주차에서 가장 흔한 실수입니다. `lab` 을 `bucket1` 로 바꿔도 AWS에는 아무 변화가 없지만, `bucket` 값을 바꾸면 버킷이 지워지고 새로 만들어집니다.

[실습] A-6. `init` : provider 받아오기 (*Initialize*)

```
terraform init
```

여기서 `provider`를 인터넷에서 받습니다. 네트워크 환경에 따라 1~3분 걸릴 수 있으니, 명령을 실행해 두고 설명을 들으세요.

[확인] `init` 성공 신호

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.57.1...
- Installed hashicorp/aws v6.57.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

셋째 줄을 A-2와 맞춰 읽으세요. `matching "~> 6.0"` : 내가 `versions.tf` 에 적은 제약이 그대로 나옵니다. 그 범위 안에서 최신인 `6.57.1` 을 선택했다고 알려줍니다.

폴더에 두 가지가 새로 생겼습니다.

새로 생긴 것	정체	커밋?
<code>.terraform/</code>	받아온 provider 실물 (수십 MB)	안 함
<code>.terraform.lock.hcl</code>	이번에 고른 정확한 버전 기록	함

! 꼭 기억

`.terraform.lock.hcl` 은 반드시 커밋합니다. `init` 출력에 그 이유가 적혀 있습니다. 이 파일이 있으면 다음에 누가 `init` 을 하든 같은 버전을 선택합니다. 없으면 오늘 나는 6.57.1로, 내일 팀원은 6.61.0으로 실행하게 되어 "제 컴퓨터에서는 되는데요"와 같은 문제가 생깁니다. `.terraform/` 는 언제나 `init` 으로 다시 받으면 되니 커밋하지 않습니다.

[실습] A-7. `fmt · validate` : apply 전에 서식과 문법 점검하기 (Author 마무리)

```
terraform fmt      # 들여쓰기·정렬 자동 교정, 고친 파일 이름을 출력
terraform validate  # 문법·정합성 검사
```

```
main.tf
```

```
Success! The configuration is valid.
```

`fmt` 는 고친 파일 이름만 출력합니다. 아무것도 안 나오면 이미 정렬이 맞다는 뜻입니다.

`validate` 는 두 가지를 봅니다. ① 문법이 맞는가 ② 코드 안에서 앞뒤가 맞는가. 오타 난 인자 이름, 없는 속성 참조, 잘못된 타입을 여기서 찾아냅니다. AWS에 아무것도 요청하지 않으므로 비용이 들지 않고 즉시 끝납니다.

참고

`validate` 는 `init` 다음에만 제대로 동작합니다. "aws_s3_bucket 에 bucket 이라는 인자가 있는가?"를 판단하려면 provider가 먼저 깔려 있어야 하기 때문입니다. 그래서 순서가 `init` → `validate` 입니다.

팁

1분만 들여 일부러 오류를 만들어 보세요. `bucket` 을 `buckett` 으로 바꾸고 `terraform validate` 를 돌리면 이렇게 나옵니다.

```
Error: Unsupported argument

on main.tf line 4, in resource "aws_s3_bucket" "lab":
  4:   buckett = "...

An argument named "buckett" is not expected here.
```

줄 번호까지 표시해 줍니다. apply 도중에 발견했다면 이미 리소스가 만들어진 뒤였을 것입니다. 확인했으면 원래대로 되돌리세요.

[실습] A-8. `plan` : 실행할 작업 미리 보기 (Plan)

```
terraform plan
```

[확인] plan 읽는 법

```
data.aws_caller_identity.current: Reading...
data.aws_caller_identity.current: Read complete after 0s [id=123456789012]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# aws_s3_bucket.lab will be created
+ resource "aws_s3_bucket" "lab" {
  + arn                = (known after apply)
  + bucket              = "boaz26-w1-kdh1834-lab-123456789012"
  + bucket_domain_name = (known after apply)
  + force_destroy       = false
  + id                  = (known after apply)
  + tags                = {
    + "Project" = "boaz26-w1-kdh1834"
    + "Study"   = "boaz-terraform-26"
    + "Week"    = "1"
  }
  ...
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

읽을 곳이 네 군데입니다.

① 맨 위 두 줄: 데이터 소스가 먼저 읽힙니다. `Read complete ... [id=123456789012]` 가 A-1에서 본 `Account` 값과 같은지 확인하세요. 데이터 소스는 **plan 단계에서 이미 읽힙니다**. 그래서 `bucket` 값이 `(known after apply)` 가 아니라 **완성된 문자열로** 표시됩니다.

② **+** 기호. 이 출력은 `git diff` 와 같은 형식입니다. 그래서 읽는 법도 같습니다.

기호	뜻
+	새로 만든다
~	고친다
-	지운다

③ `(known after apply)` . "지금은 모른다"는 뜻입니다. ARN이나 도메인 이름은 **AWS가 실제로 만들어줘야** 정해지는 값이라 plan 시점에는 알 수 없습니다. 오류가 아닙니다.

④ 맨 아랫줄 `Plan: 1 to add, 0 to change, 0 to destroy.` 항상 이 줄을 읽으세요.

! 꼭 기억

plan은 "실행 전 미리보기"이고, 아직 AWS에는 아무 변경도 일어나지 않았습니다. 숫자가 예상과 다르거나 - (삭제)가 보이면 apply하지 말고 멈추세요. 계획이 이상할 때 비용 없이 중단할 수 있는 시점은 여기뿐입니다.

💡 팁

plan 은 AWS를 읽기만 합니다. 몇 번을 실행해도 아무것도 바뀌지 않습니다. 헛갈리면 먼저 plan 을 실행하세요.

[함정] plan이 멈추거나 이상할 때

증상	원인
<code>var.project_name</code> 을 입력하라며 멈춤	<code>terraform.tfvars</code> 를 안 만들 (A-3)
No changes.	<code>main.tf</code> 의 TODO가 아직 주석 상태
Inconsistent dependency lock file	<code>init</code> 을 안 했거나, <code>versions.tf</code> 를 고치고 <code>init</code> 을 다시 안 함

Block B. 각자 apply (20분)

5단계 중 Apply, 그리고 Author → Plan → Apply 반복

[실습] B-1. apply : 실제로 만들기 (Apply)

```
terraform apply
```

apply 는 plan을 다시 계산해서 보여주고 확인을 받습니다. Terraform이 인프라를 바꿀 때 항상 계획 확인과 실행의 두 단계로 나누는 이유입니다.

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes
```

⚠️ 주의

yes 를 정확히 입력해야 진행됩니다. y , Y , Yes 모두 안 됩니다. 그리고 yes 가 아닌 다른 값을 입력하면 취소됩니다. 방금 본 plan이 이상했다면 여기서 엔터를 입력해 취소하면 됩니다. 이것이 이 프롬프트가 있는 이유입니다.

[확인] apply 성공 신호

```
aws_s3_bucket.lab: Creating...
aws_s3_bucket.lab: Creation complete after 3s [id=boaz26-w1-kdh1834-lab-123456789012]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

[id=...] 는 A-8의 plan에서 (known after apply) 였던 값입니다. 이제 값이 채워졌습니다.

! 꼭 기억

CP2: Apply complete! 줄을 project_name 과 함께 Discord에 붙여넣어 주세요. (스크린샷 말고 텍스트로)

[함정] apply가 실패할 때

에러	원인	해결
BucketAlreadyOwnedByYou	내 계정에 이미 같은 이름이 있음	이름 변경 또는 기존 버킷 정리
InvalidBucketName	대문자·언더스코어 사용, 또는 63자 초과	소문자·숫자·하이픈만, project_name 을 짧게
AccessDenied	IAM 권한 부족	실습용 사용자 권한 확인
BucketAlreadyExists	다른 사람 계정과 충돌	계정번호가 붙어 거의 안 나지만, 나면 project_name 변경

📌 참고

apply가 중간에 실패했다면 첫 행동은 terraform plan 입니다. Terraform에는 자동 롤백이 없어서 성공한 데까지는 만들어져 있고 state에도 적혀 있습니다. plan이 "남은 작업"만 다시 계산해줍니다. 코드를 전부 다시 작성하지 마세요.

[실습] B-2. state 확인하기

apply는 리소스를 만드는 것으로 끝나지 않습니다. 그 결과를 terraform.tfstate 에 기록하는 것까지 포함됩니다. 그 기록을 직접 확인합니다.

```
terraform state list
```

```
data.aws_caller_identity.current
aws_s3_bucket.lab
```

A-5에서 본 리소스 주소가 그대로 나옵니다. 그리고 데이터 소스도 목록에 있습니다. 실제 리소스가 아닌데도 Terraform이 state에 추적합니다. 다음 plan에서 "그때 읽은 값과 지금 값이 같은가"를 비교해야 하기 때문입니다.

이번엔 state 전체를 봅니다.

```
terraform show
```

```
# data.aws_caller_identity.current:
data "aws_caller_identity" "current" {
  account_id = "123456789012"
  arn        = "arn:aws:iam::123456789012:user/boaz-tf-study"
  id         = "123456789012"
  user_id    = "AIDAXXXXXXXXXXXXXXXXXX"
}

# aws_s3_bucket.lab:
resource "aws_s3_bucket" "lab" {
  arn                = "arn:aws:s3:::boaz26-w1-kdh1834-lab-123456789012"
  bucket             = "boaz26-w1-kdh1834-lab-123456789012"
  bucket_domain_name = "boaz26-w1-kdh1834-lab-123456789012.s3.amazonaws.com"
  force_destroy      = false
  id                 = "boaz26-w1-kdh1834-lab-123456789012"
  ...
}
```

데이터 소스가 A-1의 CLI와 같은 값을 가져왔다는 것을 여기서 확인할 수 있습니다.

세 명령의 차이는 다음과 같습니다.

명령	보여주는 것	언제 쓰나
<code>terraform state list</code>	주소 목록만 한 줄씩	지금 무엇을 관리 중인지 빠르게 볼 때
<code>terraform show</code>	state 전체를 사람이 읽는 형식으로	전부 다 보고 싶을 때
<code>terraform state show <주소></code>	그 한 개의 모든 속성	특정 리소스만 자세히 볼 때

```
terraform state show aws_s3_bucket.lab # 하나만 지정해서
ls -a                                  # terraform.tfstate 파일 확인
```

🚫 절대 금지

`terraform.tfstate` 를 열어보면 모든 속성이 평문 JSON입니다. 오늘은 버킷이라 민감한 값이 없지만, DB 비밀번호가 들어가는 리소스도 똑같이 평문으로 적힙니다. 그래서 `.tfstate` 는 커밋 금지입니다(`.gitignore` 에 이미 들어 있습니다). 팀이 이걸 안전하게 공유하는 방법이 Week 3 Remote Backend입니다.

[관찰 🕒] state 기록

- `terraform state list` 에 나온 줄 수와, 그중 실제 리소스는 몇 개인가:

- `state show` 에서 `arn` 값은 무엇인가: `_____`
- plan에서 `(known after apply)` 였던 값이 지금은 채워져 있나: `_____`
- `data.aws_caller_identity.current` 의 `account_id` 가 A-1의 `Account` 와 같은가: `_____`

[실습] B-3. 코드를 고치고 다시 apply (*Author* → *Plan* → *Apply* 두 번째 반복)

여기서부터가 오늘의 핵심입니다. 이번에는 아무것도 없는 상태가 아니라 이미 있는 인프라에 추가합니다.

`main.tf` 의 세 번째 TODO를 채웁니다.

```
resource "aws_s3_bucket_public_access_block" "lab" {
  bucket                = aws_s3_bucket.lab.id
  block_public_acls     = true
  block_public_policy   = true
  ignore_public_acls    = true
  restrict_public_buckets = true
}
```

그리고 `outputs.tf` 도 채웁니다.

```
output "bucket_id" {
  description = "생성된 S3 버킷 이름"
  value       = aws_s3_bucket.lab.id
}

output "bucket_arn" {
  description = "생성된 S3 버킷 ARN"
  value       = aws_s3_bucket.lab.arn
}
```

`bucket = aws_s3_bucket.lab.id` 이 한 줄을 보세요. 버킷 이름을 다시 타이핑하지 않고 다른 리소스의 주소를 참조했습니다. 이렇게 쓰면 Terraform이 "버킷을 먼저 만들고 그다음 차단 설정"이라는 순서를 스스로 압니다. 내가 순서를 어디에도 적지 않았는데 순서가 지켜집니다. 이것 **암묵적 의존성**이라 하고, Week 2의 주제입니다.

```
terraform fmt && terraform validate
terraform plan
```

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ bucket_arn = "arn:aws:s3:::boaz26-w1-kdh1834-lab-123456789012"
+ bucket_id  = "boaz26-w1-kdh1834-lab-123456789012"
```

이번 plan에서는 **bucket** 값이 `(known after apply)` 가 아닙니다. 버킷은 이미 만들어졌고 state에 적혀 있어서 Terraform이 그 값을 알기 때문입니다. 그리고 **Changes to Outputs** 는 **add/change/destroy** 숫자에 포함되지 않습니다. output은 인프라가 아니라 화면에 출력하는 값이기 때문입니다.

```
terraform apply    # yes
```

[확인] 두 번째 apply의 의미

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

```
bucket_arn = "arn:aws:s3:::boaz26-w1-kdh1834-lab-123456789012"
bucket_id  = "boaz26-w1-kdh1834-lab-123456789012"
```

버킷은 다시 만들어지지 않았습니다. 0 changed 이고 1 added 뿐입니다. 이미 원하는 상태이기 때문입니다.

이게 선언형입니다. 나는 "추가해라"라고 명령하지 않았습니다. "최종 상태는 버킷 + 차단 설정이다"라고 적었을 뿐이고, Terraform이 기록(state)과 코드(.tf)의 차이만 계산해서 실행했습니다.

[실습] B-4 (심화★). 버전 관리 켜기

main.tf 의 마지막 TODO입니다. Author → Plan → Apply 세 번째 반복입니다. 앞선 두 번의 과정과 동일합니다.

```
resource "aws_s3_bucket_versioning" "lab" {
  bucket = aws_s3_bucket.lab.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

```
terraform apply    # 또 1 to add
```

[확인] Block B 완료 기준

```
terraform state list
```

```
data.aws_caller_identity.current
aws_s3_bucket.lab
aws_s3_bucket_public_access_block.lab
aws_s3_bucket_versioning.lab      # 심화까지 했다면
```

! 꼭 기억

이 출력을 지금 복사해서 워크북에 붙여주세요. destroy 하면 복구할 수 없습니다. 과제 제출의 증빙은 이 출력뿐입니다.

[관찰 📖] Block B 기록

- 두 번째 `apply` 에서 버킷이 다시 만들어졌나, 아닌가. 왜 그런가: `_____`
- `state list` 에 나온 주소 전부: `_____`
- `Changes to Outputs` 가 `Plan: N to add` 숫자에 안 들어간 이유: `_____`

Block C. destroy (7분): 이 단계를 하지 않으면 오늘 실습은 실패입니다

[실습] C-1. 지우기

```
terraform destroy    # yes
```

`destroy` 도 plan을 먼저 보여주고 `yes` 를 받습니다. 이번엔 전부 `-` 로 나옵니다. `Plan: 0 to add, 0 to change, 3 to destroy.` 줄을 반드시 읽고 `yes` 를 치세요.

[확인] destroy 완료

```
Plan: 0 to add, 0 to change, 3 to destroy.
```

```
aws_s3_bucket_public_access_block.lab: Destroying...
aws_s3_bucket_versioning.lab: Destroying...
aws_s3_bucket_versioning.lab: Destruction complete after 0s
aws_s3_bucket_public_access_block.lab: Destruction complete after 0s
aws_s3_bucket.lab: Destroying...
aws_s3_bucket.lab: Destruction complete after 0s
```

```
Destroy complete! Resources: 3 destroyed.
```

심화(B-4)를 안 했으면 `2 destroyed` 입니다. 데이터 소스는 이 숫자에 포함되지 않습니다. 만든 적이 없으니 지울 것도 없습니다.

💡 팁

삭제 순서를 보세요. `public_access_block` 과 `versioning` 이 동시에 지워지고, 그다음에야 버킷이 지워집니다. 앞의 둘은 서로 상관없으니 병렬로, 버킷은 둘이 사라진 뒤에야 지울 수 있으니 마지막에. 만들 때의 역순입니다. B-3의 `bucket = aws_s3_bucket.lab.id` 한 줄로 생긴 의존성이 삭제 순서에도 그대로 적용됩니다.

```
terraform state list
```

! 꼭 기억

출력이 완전히 비어 있어야 성공입니다. 데이터 소스까지 함께 사라집니다. 관리할 리소스가 없으면 다시 읽어올 이유도 없기 때문입니다.

! 꼭 기억

CP3: 이 출력을 Discord에 올려주세요. 8명 전원 확인 전에는 세션을 끝내지 않습니다.

[실습] C-2. 콘솔에서 직접 확인

AWS 콘솔 → S3 → 버킷 목록에서 사라졌는지 확인합니다. state가 비었다고 끝이 아니라, 콘솔에서 실제로 삭제됐는지 확인해야 라이프사이클이 종료된 것입니다.

📌 참고

S3 버킷 목록 화면은 리전과 무관하게 전체를 보여줍니다. 콘솔 우측 상단 리전이 서울이 아니어도 버킷이 보입니다. 우리 버킷은 실제로는 `ap-northeast-2` 에 있습니다.

[실습] C-3. 커밋 사고 방지

```
git status
```

파일	이래야 정상
<code>terraform.tfvars</code>	안 보임 (<code>.gitignore</code>)
<code>terraform.tfstate</code>	안 보임 (<code>.gitignore</code>)
<code>.terraform/</code>	안 보임 (<code>.gitignore</code>)
<code>main.tf</code> , <code>outputs.tf</code>	보임 → 커밋 대상
<code>.terraform.lock.hcl</code>	보임 → 커밋 대상 (A-6)

🚫 절대 금지

`terraform.tfvars` 나 `terraform.tfstate` 가 목록에 보인다면 커밋하지 말고 멈추세요. `.gitignore` 가 적용되지 않은 상태입니다. Discord에 알려주세요.

[함정] destroy가 실패할 때

에러	원인	해결
BucketNotEmpty	버킷에 객체가 있음	객체를 비우거나 <code>force_destroy = true</code> 이해 (심화)
Error acquiring the state lock	다른 터미널에서 명령이 돌고 있음	그 명령이 끝날 때까지 대기

세션 후: 실습 결과 제출 (브랜치 → PR)

세션 중에는 `practice/` 에서 실습하면 됩니다. 세션이 끝나면 아래 절차를 진행하세요.

```
git switch main && git pull
git switch -c week1/{github-id}

mkdir -p submissions/{github-id}
cp practice/*.tf practice/example.tfvars submissions/{github-id}/
cp practice/.terraform.lock.hcl submissions/{github-id}/

git add submissions/{github-id}
git status # tfvars / tfstate 안 올라갔는지!
git commit -m "week1: {github-id} 실습 제출"
git push -u origin week1/{github-id}
```

그다음 GitHub에서 `main` 대상 PR을 올리면 강사가 리뷰 후 머지합니다.

! 꼭 기억

제출은 `submissions/{github-id}/` 본인 폴더에. `practice/` 를 직접 고쳐서 올리면 머지하는 순간 다음 사람이 풀 # TODO 가 없어집니다. 폴더를 나눠야 8명 PR이 전부 머지됩니다.

체크포인트 (Definition of Done)

- [] `terraform init` 성공, `.terraform.lock.hcl` 생성 확인
- [] `terraform apply` 성공, `state list` 에 버킷 존재
- [] 버킷 + public access block 생성 확인
- [] `terraform show` 로 데이터 소스가 읽어온 `account_id` 확인
- [] `terraform destroy` 완료 & `state list` 에 S3 리소스 없음
- [] 콘솔에서 버킷 사라진 것 확인
- [] `git status` 로 `*.tfvars` / `*.tfstate` 안 올라갔는지 확인

오늘의 명령 요약

5단계 순서에 따라 정리한 것입니다.

terraform version	# [1] Scope	: 도구 확인
	# [2] Author	: 에디터로 .tf 작성
terraform fmt	#	형식 교정
terraform init	# [3] Initialize:	provider 다운로드 (보통 한 번)
terraform validate	#	문법·정합성 검사 (init 이후)
terraform plan	# [4] Plan	: 미리보기 - 안전, AWS 안 바뀜
terraform apply	# [5] Apply	: 실행 - AWS에 씀
terraform state list	#	관리 중인 주소 목록
terraform show	#	state 전체 보기
terraform destroy	#	전부 삭제 - 세션 종료 시 필수

다음 주 예고

Week 2. 리소스 문법 & State: VPC/Subnet/EC2를 만들면서 오늘 다룬 두 가지 개념을 자세히 학습합니다.

- `bucket = aws_s3_bucket.lab.id` 한 줄로 생긴 **암묵적 의존성**이 어떻게 그래프가 되는지
- 오늘 `data "aws_caller_identity"` 로 처음 사용한 **데이터 소스**로 AMI·AZ 같은 값을 조회하는 방법
- `terraform.tfstate` 내부가 실제로 어떻게 생겼는지

예습: `resource` 참조로 생기는 의존성, `data source`, VPC/Subnet/EC2 기본 문법

참고 문서

- [What is IaC with Terraform?](#): 배포 5단계(Scope/Author/Initialize/Plan/Apply)의 출처
- [Build Infrastructure \(AWS Get Started\)](#): 이 실습의 기반이 된 문서. `terraform` 블록·`validate`·`plan` 읽는 법·`show`·데이터 소스 설명의 원본
- [Install Terraform](#): 설치가 안 돼 있으면 여기부터
- [Version Constraints](#): `~> 6.0` 이 무슨 범위인지
- [aws_s3_bucket \(Terraform Registry\)](#)
- [aws_caller_identity \(data source\)](#)

! 꼭 기억

끝났으면 `terraform destroy` 를 정말로 했는지 한 번 더 확인하세요. 삭제하지 않은 리소스는 요금이 계속 청구됩니다.